Java Concepts Cheat Sheets

1. Java Basics

Variables and Data Types

Primitive Data Types:

```java
byte b = 127;          // 8-bit, -128 to 127
short s = 32767;       // 16-bit, -32,768 to 32,767
int i = 2147483647;    // 32-bit, -2^31 to 2^31-1
long l = 9223372036854775807L; // 64-bit, -2^63 to 2^63-1
float f = 3.14f;       // 32-bit floating point
double d = 3.14159;    // 64-bit floating point
char c = 'A';          // 16-bit Unicode character
boolean bool = true;   // true or false
```

Reference Data Types:

```java
String str = "Hello World";
Integer intObj = 42;
Double doubleObj = 3.14;
```

Constants:

```java
final double PI = 3.14159;
```

Type Casting:

```java
// Widening (implicit)
int num = 10;
double decimal = num;  // int to double

// Narrowing (explicit)
double decimal = 10.5;
int num = (int) decimal;  // double to int
```

Operators

Arithmetic: `+`, `-`, `*`, `/`, `%`

```java
int sum = 5 + 3;       // 8
int diff = 5 - 3;      // 2
int product = 5 * 3;   // 15
int quotient = 5 / 3; // 1
int remainder = 5 % 3; // 2
```

Comparison: `==`, `!=`, `>`, `<`, `>=`, `<=`

```java
boolean isEqual = (5 == 3);     // false
boolean isNotEqual = (5 != 3);  // true
boolean isGreater = (5 > 3);    // true
```

Logical: `&&`, `||`, `!`

```java
boolean andResult = true && false;  // false
boolean orResult = true || false;   // true
boolean notResult = !true;          // false
```

Assignment: `=`, `+=`, `-=`, `*=`, `/=`, `%=`

```java
int x = 10;
x += 5;  // x = x + 5; (x becomes 15)
```

Increment/Decrement: `++`, `--`

```java
int count = 5;
count++;  // count is now 6
count--;  // count is now 5 again
```

2. Control Flow

Conditional Statements

If-Else:

```java
int age = 18;

if (age < 13) {
    System.out.println("Child");
} else if (age < 18) {
    System.out.println("Teenager");
} else {
    System.out.println("Adult");
}
```

Ternary Operator:

```java
String status = (age >= 18) ? "Adult" : "Minor";
```

Switch Statement:

```java
int day = 3;
String dayName;

switch (day) {
    case 1:
        dayName = "Monday";
        break;
    case 2:
        dayName = "Tuesday";
        break;
    case 3:
        dayName = "Wednesday";
        break;
    // ...other cases
    default:
        dayName = "Invalid day";
}
```

Enhanced Switch (Java 14+):

```java
String dayName = switch (day) {
    case 1 -> "Monday";
    case 2 -> "Tuesday";
    case 3 -> "Wednesday";
    // ...other cases
    default -> "Invalid day";
};
```

Loops

For Loop:

```java
for (int i = 0; i < 5; i++) {
    System.out.println(i);  // Prints 0 to 4
}
```

Enhanced For Loop (For-Each):

```java
int[] numbers = {1, 2, 3, 4, 5};
for (int num : numbers) {
    System.out.println(num);
}
```

While Loop:

```java
int i = 0;
while (i < 5) {
    System.out.println(i);
    i++;
}
```

Do-While Loop:

java

```java
int i = 0;
do {
    System.out.println(i);
    i++;
} while (i < 5);
```

Break and Continue:

java

```java
// Breaking out of a loop
for (int i = 0; i < 10; i++) {
    if (i == 5) {
        break;  // Exit loop when i equals 5
    }
    System.out.println(i);
}

// Skipping iteration
for (int i = 0; i < 10; i++) {
    if (i % 2 == 0) {
        continue;  // Skip even numbers
    }
    System.out.println(i);  // Prints only odd numbers
}
```

3. Arrays and Strings

Arrays

Array Declaration and Initialization:

java

```java
// Declaration
int[] numbers;

// Initialization
numbers = new int[5];  // Array of size 5, all elements initialized to 0

// Combined declaration and initialization
int[] numbers = new int[5];

// Declaration with values
int[] numbers = {1, 2, 3, 4, 5};
```

Accessing Array Elements:

java

```java
int firstElement = numbers[0];  // First element (index 0)
numbers[2] = 10;                // Change element at index 2
int length = numbers.length;    // Get array length
```

Multidimensional Arrays:

java

```java
// 2D array
int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};

// Accessing element
int value = matrix[1][2];  // Gets 6 (row 1, column 2)
```

Arrays Utility Class:

java

```java
import java.util.Arrays;

// Sorting
int[] numbers = {5, 2, 9, 1, 5};
Arrays.sort(numbers);  // Sorts in-place: [1, 2, 5, 5, 9]

// Binary search (on sorted array)
int index = Arrays.binarySearch(numbers, 5);  // Returns index of 5

// Fill
int[] newArray = new int[5];
Arrays.fill(newArray, 10);  // All elements become 10

// Compare
int[] array1 = {1, 2, 3};
int[] array2 = {1, 2, 3};
boolean isEqual = Arrays.equals(array1, array2);  // true

// Convert to string
String arrayString = Arrays.toString(numbers);  // "[1, 2, 5, 5, 9]"
```

Strings

String Creation:

java

```java
String s1 = "Hello";               // String literal
String s2 = new String("Hello");   // String object
```

Common String Operations:

java

```java
String str = "Hello World";

// Length
int length = str.length();  // 11

// Character access
char firstChar = str.charAt(0);  // 'H'

// Substring
String sub1 = str.substring(6);    // "World"
String sub2 = str.substring(0, 5); // "Hello"

// Concatenation
String newStr = str + "!";  // "Hello World!"
String concat = str.concat("!");  // "Hello World!"

// Equality
boolean isEqual = str.equals("Hello World");  // true
boolean ignoreCase = str.equalsIgnoreCase("hello world");  // true

// Search
int index = str.indexOf("World");  // 6
boolean contains = str.contains("Hello");  // true

// Replace
String replaced = str.replace("World", "Java");  // "Hello Java"

// Case conversion
String upper = str.toUpperCase();  // "HELLO WORLD"
String lower = str.toLowerCase();  // "hello world"

// Trimming whitespace
String withSpaces = "   Hello   ";
String trimmed = withSpaces.trim();  // "Hello"

// Split
String csv = "apple,banana,orange";
String[] fruits = csv.split(",");  // ["apple", "banana", "orange"]
```

String Builder (Mutable Strings):

java

```java
StringBuilder sb = new StringBuilder();
sb.append("Hello");
sb.append(" ");
sb.append("World");
String result = sb.toString();  // "Hello World"

// Other operations
sb.insert(5, ",");  // "Hello, World"
sb.delete(5, 6);    // "HelloWorld"
sb.reverse();       // "dlroW olleH"
```

4. Methods

Method Declaration:

java

```java
// Basic method
public void greet() {
    System.out.println("Hello!");
}

// Method with parameters
public void greetPerson(String name) {
    System.out.println("Hello, " + name + "!");
}

// Method with return value
public int add(int a, int b) {
    return a + b;
}

// Method with multiple parameters
public double calculateAverage(double[] numbers) {
    double sum = 0;
    for (double num : numbers) {
        sum += num;
    }
    return sum / numbers.length;
}
```

Method Overloading:

java

```java
// Methods with same name but different parameters
public int multiply(int a, int b) {
    return a * b;
}

public double multiply(double a, double b) {
    return a * b;
}

public int multiply(int a, int b, int c) {
    return a * b * c;
}
```

Parameter Passing:

java

```java
// Pass by value (primitives)
public void incrementValue(int x) {
    x = x + 1;  // Only modifies local copy
}

int number = 5;
incrementValue(number);  // number still equals 5

// Pass by reference (object references)
public void modifyArray(int[] arr) {
    arr[0] = 100;  // Modifies the original array
}

int[] myArray = {1, 2, 3};
modifyArray(myArray);  // myArray is now {100, 2, 3}
```

Variable Arguments (Varargs):

java

```java
public int sum(int... numbers) {
    int total = 0;
    for (int num : numbers) {
        total += num;
    }
    return total;
}

// Can be called with any number of arguments
int result1 = sum(1, 2);           // 3
int result2 = sum(1, 2, 3, 4, 5);  // 15
```

Recursion:

java

```java
public int factorial(int n) {
    if (n <= 1) {
        return 1;
    }
    return n * factorial(n - 1);
}
```

5. Object-Oriented Programming

Classes and Objects

Class Declaration:

java

```java
public class Person {
    // Fields (attributes)
    private String name;
    private int age;

    // Constructor
    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    // Default constructor
    public Person() {
        this.name = "Unknown";
        this.age = 0;
    }

    // Methods
    public void sayHello() {
        System.out.println("Hello, my name is " + name);
    }

    // Getters and Setters
    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        if (age >= 0) {
            this.age = age;
        }
    }
}
```

Creating Objects:

java

```java
// Using constructor
Person person1 = new Person("John", 30);

// Using default constructor
Person person2 = new Person();
person2.setName("Jane");
person2.setAge(25);

// Accessing object methods
person1.sayHello();
String name = person1.getName();
```

Static Members:

java

```java
public class MathUtils {
    // Static variable (shared among all instances)
    public static final double PI = 3.14159;

    // Static method
    public static int add(int a, int b) {
        return a + b;
    }
}

// Using static members (without creating an object)
double area = MathUtils.PI * radius * radius;
int sum = MathUtils.add(5, 3);
```

The `this` Keyword:

java

```java
public class Counter {
    private int count;

    public void increment() {
        this.count++;  // 'this' refers to current object
    }

    public Counter getThis() {
        return this;  // Returns reference to current object
    }
}
```

Inheritance

Base Class and Derived Class:

java

```java
// Base class (parent)
public class Animal {
    protected String name;

    public Animal(String name) {
        this.name = name;
    }

    public void eat() {
        System.out.println(name + " is eating");
    }

    public void sleep() {
        System.out.println(name + " is sleeping");
    }
}

// Derived class (child)
public class Dog extends Animal {
    private String breed;

    public Dog(String name, String breed) {
        super(name);  // Call to parent constructor
        this.breed = breed;
    }

    public void bark() {
        System.out.println(name + " is barking");
    }

    // Method overriding
    @Override
    public void eat() {
        System.out.println(name + " the " + breed + " is eating dog food");
    }
}
```

Using Inherited Classes:

java

```java
Animal myAnimal = new Animal("Generic Animal");
myAnimal.eat();  // "Generic Animal is eating"

Dog myDog = new Dog("Rex", "German Shepherd");
myDog.eat();    // "Rex the German Shepherd is eating dog food" (overridden)
myDog.sleep();  // "Rex is sleeping" (inherited)
myDog.bark();   // "Rex is barking" (specialized)
```

Super Keyword:

java

```java
public class Cat extends Animal {
    public Cat(String name) {
        super(name);  // Call parent constructor
    }

    @Override
    public void eat() {
        super.eat();  // Call parent's method
        System.out.println("and purring");
    }
}
```

Polymorphism

Method Overriding:

java

```java
public class Shape {
    public double calculateArea() {
        return 0;
    }
}

public class Circle extends Shape {
    private double radius;

    public Circle(double radius) {
        this.radius = radius;
    }

    @Override
    public double calculateArea() {
        return Math.PI * radius * radius;
    }
}

public class Rectangle extends Shape {
    private double width;
    private double height;

    public Rectangle(double width, double height) {
        this.width = width;
        this.height = height;
    }

    @Override
    public double calculateArea() {
        return width * height;
    }
}
```

Polymorphic Usage:

```java
// Reference type is parent, object type is child
Shape shape1 = new Circle(5);
Shape shape2 = new Rectangle(4, 6);

// Method called depends on actual object type
double area1 = shape1.calculateArea();  // Uses Circle's implementation
double area2 = shape2.calculateArea();  // Uses Rectangle's implementation

// Using polymorphism in arrays/collections
Shape[] shapes = {new Circle(3), new Rectangle(2, 4), new Circle(5)};
for (Shape shape : shapes) {
    System.out.println("Area: " + shape.calculateArea());
}
```

Downcasting:

```java
Shape shape = new Circle(10);

// Check before casting
if (shape instanceof Circle) {
    Circle circle = (Circle) shape;  // Downcast to access Circle-specific methods
}
```

Abstraction

Abstract Classes:

java

```java
public abstract class Vehicle {
    protected String brand;

    // Constructor
    public Vehicle(String brand) {
        this.brand = brand;
    }

    // Regular method
    public void start() {
        System.out.println("The vehicle is starting");
    }

    // Abstract method (must be implemented by subclasses)
    public abstract void move();
}

public class Car extends Vehicle {
    public Car(String brand) {
        super(brand);
    }

    @Override
    public void move() {
        System.out.println("The car is driving on the road");
    }

    // Car-specific method
    public void honk() {
        System.out.println("Beep beep!");
    }
}
```

Interfaces:

java

```java
public interface Flyable {
    // Constants (implicitly public, static, final)
    int MAX_ALTITUDE = 10000;

    // Abstract methods (implicitly public abstract)
    void fly();
    void land();

    // Default method (Java 8+)
    default void takeOff() {
        System.out.println("Taking off...");
    }

    // Static method (Java 8+)
    static boolean canFly(Object obj) {
        return obj instanceof Flyable;
    }
}

public class Bird extends Animal implements Flyable {
    public Bird(String name) {
        super(name);
    }

    @Override
    public void fly() {
        System.out.println(name + " is flying with wings");
    }

    @Override
    public void land() {
        System.out.println(name + " is landing on a tree");
    }
}

public class Airplane implements Flyable {
    private String model;

    @Override
    public void fly() {
        System.out.println("Airplane is flying with engines");
    }

    @Override
    public void land() {
        System.out.println("Airplane is landing on runway");
```

```
    }
}
```

Using Interfaces:

```java
// Reference by interface type
Flyable flyer1 = new Bird("Eagle");
Flyable flyer2 = new Airplane();

flyer1.fly();  // "Eagle is flying with wings"
flyer2.fly();  // "Airplane is flying with engines"

// Using default methods
flyer1.takeOff();  // "Taking off..."

// Using static methods
boolean canBirdFly = Flyable.canFly(flyer1);  // true
```

Multiple Interface Implementation:

```java
public interface Swimming {
    void swim();
}

public class Duck extends Bird implements Swimming {
    public Duck(String name) {
        super(name);
    }

    @Override
    public void swim() {
        System.out.println(name + " is swimming");
    }
}
```

6. Exception Handling

Types of Exceptions:

- `Throwable` - Base class for all errors and exceptions
  - `Error` - Serious problems, should not be caught (e.g., `OutOfMemoryError`)

- `Exception` - Problems that can be handled
  - Checked exceptions - Must be declared or caught (e.g., `IOException`)
  - Unchecked exceptions - Runtime exceptions (e.g., `NullPointerException`)

Try-Catch Block:

java

```java
try {
    // Code that might throw an exception
    int result = 10 / 0;  // Will throw ArithmeticException
    System.out.println("This won't execute");
} catch (ArithmeticException e) {
    // Handle specific exception
    System.out.println("Cannot divide by zero");
} catch (Exception e) {
    // Catch-all for other exceptions
    System.out.println("Something went wrong: " + e.getMessage());
} finally {
    // Always executes, regardless of exception
    System.out.println("Finally block always runs");
}
```

Try-with-Resources (Java 7+):

java

```java
// Resources are automatically closed
try (
    FileInputStream input = new FileInputStream("file.txt");
    BufferedReader reader = new BufferedReader(new InputStreamReader(input))
) {
    String line = reader.readLine();
    System.out.println(line);
} catch (IOException e) {
    System.out.println("Error reading file: " + e.getMessage());
}
// No need for finally block to close resources
```

Throwing Exceptions:

```java
public void withdraw(double amount) throws InsufficientFundsException {
    if (amount > balance) {
        throw new InsufficientFundsException("Not enough funds");
    }
    balance -= amount;
}
```

Custom Exceptions:

```java
// Checked exception
public class InsufficientFundsException extends Exception {
    public InsufficientFundsException(String message) {
        super(message);
    }
}

// Unchecked exception
public class InvalidInputException extends RuntimeException {
    public InvalidInputException(String message) {
        super(message);
    }
}
```

Exception Chaining:

```java
try {
    // Some code
} catch (IOException e) {
    throw new ApplicationException("Failed to process data", e);
    // Original exception is preserved as the cause
}
```

7. Collections Framework

Collection Hierarchy

- **Collection Interface** (root)
  - **List Interface** (ordered, allows duplicates)
    - ArrayList

    - LinkedList
    - Vector
  - **Set Interface** (no duplicates)
    - HashSet
    - LinkedHashSet
    - TreeSet
  - **Queue Interface** (FIFO)
    - LinkedList
    - PriorityQueue
  - **Deque Interface** (double-ended queue)
    - ArrayDeque
    - LinkedList
- **Map Interface** (key-value pairs)
  - HashMap
  - LinkedHashMap
  - TreeMap
  - Hashtable

Lists

ArrayList:

java

```java
import java.util.ArrayList;
import java.util.List;

// Create list
List<String> names = new ArrayList<>();

// Add elements
names.add("Alice");
names.add("Bob");
names.add("Charlie");

// Access elements
String first = names.get(0);  // "Alice"

// Size
int size = names.size();  // 3

// Iteration
for (String name : names) {
    System.out.println(name);
}

// Remove elements
names.remove(1);  // Removes "Bob"
names.remove("Charlie");  // Removes by value

// Check if element exists
boolean hasAlice = names.contains("Alice");  // true

// Clear all elements
names.clear();
```

LinkedList:

java

```java
import java.util.LinkedList;

LinkedList<Integer> numbers = new LinkedList<>();

// Additional operations for LinkedList
numbers.addFirst(1);  // Add at the beginning
numbers.addLast(10);  // Add at the end

int first = numbers.getFirst();  // Get first element
int last = numbers.getLast();    // Get last element

numbers.removeFirst();  // Remove first element
numbers.removeLast();   // Remove last element
```

Sets

HashSet:

java

```java
import java.util.HashSet;
import java.util.Set;

// Create set
Set<String> uniqueNames = new HashSet<>();

// Add elements (duplicates are ignored)
uniqueNames.add("Alice");
uniqueNames.add("Bob");
uniqueNames.add("Alice");  // Ignored (duplicate)

// Size
int size = uniqueNames.size();  // 2

// Check if element exists
boolean hasBob = uniqueNames.contains("Bob");  // true

// Remove element
uniqueNames.remove("Alice");

// Iteration (order not guaranteed)
for (String name : uniqueNames) {
    System.out.println(name);
}
```

TreeSet (Sorted Set):

```java
import java.util.TreeSet;
import java.util.Set;

// Create sorted set
Set<String> sortedNames = new TreeSet<>();

// Add elements
sortedNames.add("Charlie");
sortedNames.add("Alice");
sortedNames.add("Bob");

// Iteration (automatically sorted)
for (String name : sortedNames) {
    System.out.println(name);  // Alice, Bob, Charlie
}
```

Maps

HashMap:

java

```java
import java.util.HashMap;
import java.util.Map;

// Create map
Map<String, Integer> ages = new HashMap<>();

// Add key-value pairs
ages.put("Alice", 25);
ages.put("Bob", 30);
ages.put("Charlie", 35);

// Access value by key
int aliceAge = ages.get("Alice");  // 25
Integer unknownAge = ages.get("Unknown");  // null

// Check if key/value exists
boolean hasAlice = ages.containsKey("Alice");  // true
boolean has30 = ages.containsValue(30);        // true

// Get default value if key not found
int davidAge = ages.getOrDefault("David", 0);  // 0

// Size
int size = ages.size();  // 3

// Remove entry
ages.remove("Bob");

// Iteration through entries
for (Map.Entry<String, Integer> entry : ages.entrySet()) {
    System.out.println(entry.getKey() + ": " + entry.getValue());
}

// Iteration through keys
for (String name : ages.keySet()) {
    System.out.println(name);
}

// Iteration through values
for (Integer age : ages.values()) {
    System.out.println(age);
}
```

TreeMap (Sorted Map):

java

```java
import java.util.TreeMap;
import java.util.Map;

// Create sorted map (keys are sorted)
Map<String, Integer> sortedAges = new TreeMap<>();

sortedAges.put("Charlie", 35);
sortedAges.put("Alice", 25);
sortedAges.put("Bob", 30);

// Iteration through entries (sorted by key)
for (Map.Entry<String, Integer> entry : sortedAges.entrySet()) {
    System.out.println(entry.getKey() + ": " + entry.getValue());
    // Output: Alice: 25, Bob: 30, Charlie: 35
}
```

Queue and Deque

Queue:

java

```java
import java.util.LinkedList;
import java.util.Queue;

// Create queue
Queue<String> queue = new LinkedList<>();

// Add elements
queue.offer("First");   // Add to the queue
queue.offer("Second");
queue.offer("Third");

// Peek at first element without removing
String first = queue.peek();  // "First"

// Remove and return first element
String removed = queue.poll();  // "First"
```

Priority Queue:

java

```java
import java.util.PriorityQueue;
import java.util.Queue;

// Natural ordering (smallest first)
Queue<Integer> priorityQueue = new PriorityQueue<>();

priorityQueue.offer(10);
priorityQueue.offer(5);
priorityQueue.offer(15);

// Elements are retrieved in priority order
int highest = priorityQueue.poll();  // 5 (lowest value has highest priority)
```

Deque (Double-Ended Queue):

java

```java
import java.util.ArrayDeque;
import java.util.Deque;

Deque<String> deque = new ArrayDeque<>();

// Add elements at both ends
deque.offerFirst("First");
deque.offerLast("Last");

// Peek at elements from both ends
String peekFirst = deque.peekFirst();  // "First"
String peekLast = deque.peekLast();    // "Last"

// Remove elements from both ends
String removeFirst = deque.pollFirst();  // "First"
String removeLast = deque.pollLast();    // "Last"

// Using deque as a stack
deque.push("A");  // Add to front
deque.push("B");
String top = deque.pop();  // "B" (LIFO order)
```

Utility Classes

Collections Class:

java

```java
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

List<Integer> numbers = new ArrayList<>();
numbers.add(3);
numbers.add(1);
numbers.add(5);

// Sorting
Collections.sort(numbers);  // [1, 3, 5]

// Reverse
Collections.reverse(numbers);  // [5, 3, 1]

// Shuffle
Collections.shuffle(numbers);  // Random order

// Binary search (on sorted list)
Collections.sort(numbers);
int index = Collections.binarySearch(numbers, 3);

// Find min/max
int min = Collections.min(numbers);
int max = Collections.max(numbers);

// Fill
Collections.fill(numbers, 0);  // All elements become 0

// Frequency
int count = Collections.frequency(numbers, 0);  // Count occurrences

// Unmodifiable views
List<Integer> immutableList = Collections.unmodifiableList(numbers);
```

8. Generics

Generic Classes:

java

```java
// Generic class with type parameter T
public class Box<T> {
    private T content;

    public Box(T content) {
        this.content = content;
    }

    public T getContent() {
        return content;
    }

    public void setContent(T content) {
        this.content = content;
    }
}

// Using the generic class
Box<String> stringBox = new Box<>("Hello");
String str = stringBox.getContent();  // No casting needed

Box<Integer> intBox = new Box<>(42);
Integer num = intBox.getContent();
```

Generic Methods:

java

```java
// Generic method with type parameter T
public <T> void printArray(T[] array) {
    for (T element : array) {
        System.out.println(element);
    }
}

// Using the generic method
String[] strings = {"Hello", "World"};
Integer[] integers = {1, 2, 3};

printArray(strings);
```